

INFM-PROJ-ML

EFFICIENT METHODS FOR MACHINE LEARNING

at *uni-hamburg*

with *Prof. Dr. Sören Laue* in the SoSe 2024

Report by

<i>Flavia Macovei</i>	<i>Arian Urdu</i>
7610738	1234567

FACULTY
OF MATHEMATICS, INFORMATICS
AND NATURAL SCIENCES

Abstract

This report explores strategies for enhancing the efficiency of natural language processing under constraints of limited training data and computational resources. Traditionally, “bigger” language models, i.e. models with more parameters produce better predictions than their smaller counterparts. This approach of scaling up models usually involves large datasets and powerful hardware, both of which can be expensive. The aim of this report is to analyse methods of optimisation that create a powerful model while requiring less data and processing power. This quality-over-quantity approach calls for high-caliber data in a distinct area. The dataset chosen for this work is the TinyStories corpus by Eldan and Li which is advantageous due to its fusion of diversification and linguistic simplicity. Among the optimisation techniques used in the model training process are thorough data preprocessing, a custom-made tokeniser and a curriculum learning agenda. In this report, these techniques are described and analysed with respect to their influence on training efficiency. In addition, the evaluation pipeline used in this process is demonstrated and justified. Our findings indicate that when designing a language model, size is not the only factor that can enhance prediction quality and by implementing various optimisation techniques, small language models can generate remarkably coherent text.

1 Introduction

A system which seeks to reproduce natural language has to consider a multitude of attributes: aside from providing an appropriate vocabulary, words need to be strung together coherently and follow the appropriate grammar. Each sentence observes its intrinsic logic and successive sentences need to have an underlying correlation that is coherent. In the context of natural language processing, it appears intuitive that a language model of higher complexity can fulfill these criteria better.

In this report, we consider alternative methods for improving the efficiency of a model and aim to maximise prediction quality with strict constraints on dataset size and hardware power. More specifically, the goal of this project is to develop a small language model that is trained on a curated, small-scale dataset and whose training process can be conducted on a standard laptop. The limited size of the dataset entails a need for specialisation of the prediction domain, so we aim to achieve high prediction accuracy in a specific target area. The model is to be designed in a way that it takes an input prompt in text form and predicts its continuation.

Eldan and Li pose a similar question to this report’s objective in their paper which first introduced the TinyStories dataset: “Language models ... often struggle to produce coherent and fluent text when they are small. ... This raises the question of whether the emergence of the ability to produce coherent English text only occurs at larger scales.” The TinyStories dataset consists of roughly 2 million short stories generated with GPT-3.5 and GPT-4 by OpenAI in simple English and lends itself to be used in the scope of this project. Eldan and Li argue that natural language prediction has to meet two criteria to be considered coherent: it should fit both the content and the syntax of its context.

The first suggests a form of *logical reasoning* ability of the model. Take for an example the sentence “Jessica was cold so she put on ...”. The expected completion generated by a language model should relate to a warm item of clothing. The second implies the ability to use proper grammar: not any phrase that relates to clothing can be used as a continuation of the above prompt, but only those that adhere to *grammatical and syntactical rules* of the English language.

Building on these criteria, the objectives of this report are as follows:

Reproducing stories

The model should be able to generate stories similar to those of its training set given a short prompt or no prompt. They should resemble the training stories in length, language and plot complexity. A model which achieves this objective demonstrates the ability to string words together coherently, fulfilling the *grammatical and syntactical requirement*. Additionally, it is desired that the model display a certain degree of creativity and not directly replicate its training data.

Logical inference

Secondly, a successful model is expected to be able to generate a prediction that fits the logic and content of its context. This comes into play when the input prompt is longer and provides more context, thereby restricting the realm of feasible predictions. This objective feeds into the requirement for *logical conformity* mentioned above.

Competitive with leading models

Finally, the model outlined in this project is desired to demonstrate competitive performance when compared to leading benchmarks. Specifically, it should perform as well as or better than the models presented in the TinyStories paper or other notable models on selected evaluation metrics. This comparison is an indication of the model's overall capability and of the validity of the methods presented in this report.

2 The Dataset

The TinyStories dataset introduced by Eldan and Li comprises a little over 2 million stories of about 175 words length on average. They are made up of roughly 17,000 unique words*. For reference, the average adult has a vocabulary between 25,000 and 35,000 words. Children tend to acquire an extensive vocabulary later in their linguistic development, after they already have solid foundations in grammar and reasoning. This is the rationale behind this dataset, which aims to incorporate all the elementary components of the English language without introducing excessive complexity.

The stories are generated using the GPT-3.5 and GPT-4 models and mimic children's stories with a target age of 3-4.

Each story must meet the following requirements:

- it includes three words randomly chosen from a set of 1500 representing a child's vocabulary
- the format and content follows that of a standard short story intended for children
- from a range of features such as dialogue or moral value, the story must incorporate a subset

These components ensure a definite dataset in a distinct target area but which still retains sufficient data diversification for comprehensive training. The prose nature of the data also aligns well with the core objective of language models, which is to process natural language.

It is worth mentioning that the model designed in this report relies on this single data source for its training.

While this approach might be less common, it is well-suited to this scenario because our goal is to generate in a restricted target area that is accurately represented by the data used. Moreover, using a sole source of data feeds into the need for simplicity and allows for a more streamlined process of development.

* number obtained by counting all words from the dataset that are also found in the nltk corpus of words of the English language

3 The Model

Building on the transformer architecture, the model developed for this project integrates various optimisation procedures such as thorough data preprocessing or curriculum learning. These optimisations are aimed at enhancing the model's efficiency and accuracy, particularly in generating coherent text within the targeted domain. Additionally, the model's performance is monitored through a set of evaluation metrics, ensuring that it meets the predefined benchmarks. The following section provides a detailed breakdown of these techniques and their impact on the model's performance.

Architecture

The training and prediction process of the model occurs using a transformer architecture. Transformers are a type of neural network architecture which process sequential data using self-attention mechanisms. This allows them to consider the importance of each part of the input and capture complex dependencies within it. For this reason, transformers are particularly effective for natural language processing tasks.

A 7-layer model proved to be the optimal balance between low model complexity and prediction accuracy. This configuration provides sufficient depth to capture the patterns present in the data compared to shallower models. Those struggle to generalise on the seen data and lead to results. Deeper models not only require more memory and processing power, they also produce results ... which indicates overfitting. For a limited dataset such as the one used in this project, it is crucial to monitor the model's tendency to overfit, as memorising training data can happen much easier than when using a larger dataset.

Two embedding layers transform the input data into numerical format: A *token embedding table* is used to convert the input tokens into vector representations and a layer using a *position embedding table* encodes the position of each token in the sequence.

Self-attention layers are the core component of the transformer model. They allow it to understand the dependencies within the input sequence by weighing the relation of each pair of tokens. The model utilises 8 attention heads which perform the following functions: each head projects the input sequence into query, key, and value representations, each with outputs of dimension 8. Additionally, a dropout with probability 0.3 is performed. This final step aims to prevent overfitting by randomly deactivating a fraction of the attention weights, allowing the model to become more robust against overfitting.

Following on from there, *feed-forward* layers apply linear transformations and non-linear activation to the output from the preceding self-attention layer. These contribute to the model's ability to extract complex patterns from the input data. The first linear transformation increases the dimensionality of the input's representation to 256 features. Subsequently, a ReLU (Rectified Linear Unit) function is applied, introducing non-linearity into the

model. The features are then reduced back to their original number of 64, followed by a dropout with probability of 0.3 in this layer also.

Parameters

- > embedding size
- > number of parameters
- > optimiser
- > learning rate schedule
- > loss function

Data

The dataset used in the training process consists of short stories from the TinyStories collect described above. While the raw data is highly valuable, it contains minor inconsistencies due to its generation process with a large language model, necessitating some preprocessing to ensure optimal quality for this use case.

The elementary data transformation process consisted of cleaning up those inconsistencies. Empty stories were deleted and through usage of a *set* data structure, exact duplicates were removed. Subsequently, we replaced falsely encoded symbols such as “Ã©” with their corresponding UTF-8 representation, in this case “é”*. In some instances this was not possible, and the story retained special characters despite replacing the most common encodings. Those stories were removed entirely.

This cleaning process resulted in a reduction to about 85% of the original data. The working dataset contains roughly 1.8 million data points, the number of unique words remains largely unchanged. It might appear as though this reduction performed on an already comparatively small dataset is so strong as to hinder a meaningful learning process. However, it should be pointed out that this procedure was carried out with minimal alterations, aiming to preserve the integrity of the original data. Any data that was deleted was not viable for use, therefore the reduction of *functional* data is significantly smaller. Taking this into account, the smaller size is reasonable considering this cleaning process provides high-quality data.

A more sophisticated step in the preprocessing pipeline was the creation of a three-way split on the training data for usage in the curriculum learning process which is described in detail in a later section. Such a split necessitated a ranking system for the stories resulting in a total ordering. The curriculum would then be created by sorting the stories according to their scores and dividing the resulting selection into three segments. The close connection between training data and prediction purpose suggests that the same scoring technique be used for ranking the training data as for evaluating the model output. This also has the advantage that only one scoring pipeline is implemented. The exact composition of this pipeline is examined in the *performance* section. In short, each story is passed to a large language model along with the instruction to give it a rating between 1 and 3 for its grammar, spelling, and consistency. The idea behind this automated evaluation pipeline comes from Eldan and Li, who used it to streamline the scoring process of the outputs generated by their models.

Creating this total ordering on the training data was a complex process that ran for multiple days on a powerful server. Despite the project’s emphasis on minimising hardware demands, this one-time computational investment was a necessary trade-off to facilitate optimisation at later stages.

Another important consideration when working with natural language is the choice of tokeniser. The *BasicEnglish* tokeniser by *torchtext* provides a good starting point for initial experiments, but proves too complex for the scope of this project. Any large tokeniser induces longer processing times during training, since more calculations need to be performed at every level.

Instead, we wrote a custom tokeniser to optimally handle the needs of this model. It is based on the byte-pair encoding (bpe) format and contains 4096 tokens in its vocabulary. Bpe tokenisers are beneficial in this use case compared to other types such as WordPiece or Unigram tokenisers because of their flexibility regarding out-of-vocabulary words. WordPiece and Unigram tokenisers attempt to split unknown words into sub-words contained in their vocabulary and may use a special *<unknown>* token if they are unsuccessful. Bpe tokenisers more effectively handle out-of-vocabulary words by breaking it down into subword components that exist in its vocabulary, ensuring that no part of the word is left unrepresented. This feature was critical for the choice of tokeniser, considering the chosen vocabulary size is small compared to the number of words appearing in the training data.

* see appendix for a full list of encoding replacements.

Training

- > batch size
- > number of epochs
- > training time
- > hardware specifications
- > most importantly: curriculum learning + layer freezing

Performance

- > evaluation metrics
- > plot score against loss
- > benchmarking

4 Results

Discussion of the demo:

temperature and its surprising effects

how it reacts to unknown words that could be words

how it reacts to jibberish

→ refer to appendix

- Some introduction to which of the techniques from the section above proved most useful.
1. preprocessing
 2. tokenisers
 3. curriculum learning + layer freezing

5 Future Work

cosine similarity
layer freezing schedule
ending sequences

6 Conclusion

reiterate objectives
reiterate what we did
challenges that were overcome: preprocessing large dataset, loss stagnation(, learning rate)
outstanding challenges: see future work section
say for each objective if it was met
optimistic view that small models can achieve great things